

Backpropagation, derived and coded

Tuesday's network, run backwards. For every one of its thirteen numbers we want the slope of the loss — and we get all of them from the chain rule, one layer at a time, reusing what the forward pass already computed. Then we prove the derivation right with the numerical checker you kept from Lab 6.

Unit 2 · From scratch

80 minutes

Lab 10 · three blanks

The hardest session of the term

The goal

One weight

All at once

Gradient check

Lab 10

Before the midterm

PART 1 · 10 MINUTES

What we want, and why the obvious way is too slow

1

Gradient descent needs the slope of the loss with respect to every parameter. You already own a way to get one: `numgrad` from Lab 6 — nudge a weight up, nudge it down, compare the loss. For thirteen parameters that is 26 forward passes per step. For the 1,057 parameters of session 16's network it is over two thousand, every step, thousands of steps. Real networks have millions.

BACKPROPAGATION IN ONE SENTENCE

Compute every gradient in **one backward sweep** — about the cost of one extra forward pass — by applying the chain rule from the loss back towards the inputs and **reusing** the intermediate results the forward pass already stored.

One weight, derived completely

Tuesday's example with label $y = 1$. Its loss is $-\log(0.696) = 0.362$.

2

Start at the end. For a sigmoid with cross-entropy we showed in session 11 that the slope with respect to the output score is short:

STEP 1 — THE OUTPUT SCORE

$$\partial L / \partial Z_2 = A_2 - y = 0.696 - 1 = -0.304$$

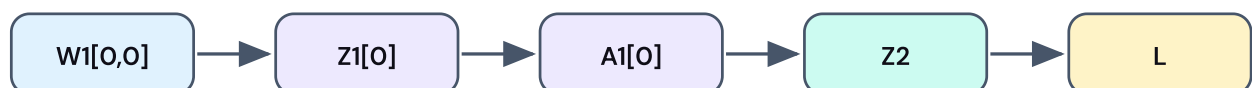
Now $W_2[0, 0]$. It enters the loss only through $Z_2 = A_1 \cdot W_2 + b_2$, multiplied by $A_1[0] = 0.971$. The chain rule multiplies the two slopes:

STEP 2 — A SECOND-LAYER WEIGHT

$$\partial L / \partial W_2[0, 0] = \partial L / \partial Z_2 \cdot A_1[0] = -0.304 \times 0.971 = -0.295$$

Negative, so descent will *increase* this weight — sensible, since neuron 1 is on and the answer should have been yes. Now a weight in the first layer, $W_1[0, 0]$. Its path to the loss is longer: it feeds $Z_1[0]$, which becomes $A_1[0]$ through a sigmoid, which reaches Z_2 through $W_2[0, 0]$.

THE CHAIN RULE IS A PRODUCT ALONG THE PATH — ONE FACTOR PER ARROW



$$\times x[0] = 2.0$$

$$\times \sigma' = 0.0285$$

$$\times W_2[0, 0] = 1.0$$

$$\times (A_2 - y) = -0.304$$

$$\partial L / \partial W_1[0, 0] = 2.0 \times 0.0285 \times 1.0 \times (-0.304) = -0.0173$$

Four local slopes, multiplied. The sigmoid's factor is $A1[0](1 - A1[0]) = 0.971 \times 0.029 = 0.0285$, and it is tiny because neuron 1 is nearly saturated — hold that thought for session 19. The result, **-0.0173**, is what `numgrad` gives too: -0.01728.

PART 3 · 15 MINUTES

From one weight to all of them: four lines

Replace the single path with matrices, and let the shapes check the algebra.

3

Every path through the network has the same structure, so we compute the shared factors once, as arrays. Moving backwards, each quantity is the one after it times one local slope:

backpropagation for the 2-3-1 network

PYTHON

```
dZ2 = (A2 - y) / m # (m, 1) slope at the output score, averaged over m
examples
dW2 = A1.T @ dZ2; db2 = dZ2.sum(axis=0) # (3, 1), (1,)
dZ1 = (dZ2 @ W2.T) * A1 * (1 - A1) # (m, 3) back through W2, then through the
sigmoid
dW1 = X.T @ dZ1; db1 = dZ1.sum(axis=0) # (2, 3), (3,)
```

THE SHAPE RULE THAT CATCHES ALMOST EVERY MISTAKE

Every gradient has exactly the shape of the thing it is the gradient of. `dW1` must be (2, 3) because `W1` is. If a line gives (3, 2), a transpose is in the wrong place; if broadcasting silently gives (4, 4), a `keepdims` is missing. Print shapes before you print values.

Forward line	Backward line	Why
<code>Z1 = X @ W1 + b1</code>	<code>dW1 = X.T @ dZ1</code>	a weight's slope is its input times the slope after it
<code>A1 = σ(Z1)</code>	<code>dZ1 = dA1 * A1 * (1 - A1)</code>	multiply by the sigmoid's local slope, entry by entry
<code>Z2 = A1 @ W2 + b2</code>	<code>dA1 = dZ2 @ W2.T</code>	blame flows back along the same weights, transposed

That table is the whole mathematics. What remains is bookkeeping — transposes, sums over the batch, `keepdims` — and bookkeeping is exactly where a derivation goes wrong without anyone noticing. So we check it.

Gradient checking: prove your derivation

The numerical slope from Lab 6, run once per parameter.

4

what the provided checker does

TEXT

```
for each parameter  $\theta$ :
    numeric = (L( $\theta$  + h) - L( $\theta$  - h)) / (2h)          # your numgrad, h = 1e-5
    analytic = the value backprop computed
    worst = max(worst, |numeric - analytic|)
```

On Lab 10's XOR network with seed-7 starting weights, a correct backward pass disagrees with the numerical slopes by about 8×10^{-12} — rounding error. Delete the sigmoid factor from `dz1` and the disagreement jumps by many orders of magnitude. And one gradient step with learning rate 1.0 lowers the loss from 0.7002 to 0.6955: the derivation is not just consistent, it points downhill.



Why not always use numgrad?

It is slow — two forward passes per parameter — and it is approximate. Backprop is fast and exact. The checker is used *once*, to earn the right to trust the fast method, and then switched off. Every serious framework was built and tested this way.

Three questions

The second one is on the midterm in some form.

Q1. Why is the sigmoid factor for neuron 1 (0.0285) so much smaller than for neuron 2 (0.149)? $\sigma' = A(1 - A)$ is largest at $A = 0.5$. Neuron 1 sits at 0.971, nearly saturated, so its slope is almost flat and very little gradient passes back through it.

Q2. What shape is `dz1` for a batch of 4 examples, and why? (4, 3): one row per example, one column per hidden neuron — the same shape as `Z1`.

Q3. The gradient check fails with a disagreement of 0.02. Name the first thing you print. The shape of every gradient against its parameter. A wrong transpose or a missing division by m is far more likely than wrong calculus.

The backprop skeleton

Lab 10 — `CSC4315_Lab10_Backprop_Skeleton`. The forward pass and half the backward pass are written. You fill three lines — `dA1`, `dZ1`, `dW1` — and the gradient check, written for you, prints PASS or FAIL. A skeleton, never a blank page.

Before the midterm

Unit 2 ends here, and so does the material for Tuesday's examination. The midterm covers units 0 to 2: half conceptual — the two-question test, the three paradigms, the history, what learning means — and half computational.

1. **Hand-compute a forward pass** through a small network, every intermediate shown.
2. **Derive one gradient** along a path, as in Part 2 today.
3. **Read and fix broken NumPy** — shapes, transposes, broadcasting.
4. **Explain, in a sentence each:** feature, label, parameter, loss, gradient, learning rate, activation, forward pass, backpropagation. Lab 10's extension B is the checklist.



Session 15 — Tuesday 27 October, 16:00 · Midterm examination

No laptop. A calculator is allowed. Then session 16, on Friday 30 October: the complete training loop, and your network meets 1,347 real handwritten digits.